Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО».

Факультет программной инженерии и компьютерной техники

Распределенные системы хранения данных
Лабораторная работа №1
Вариант №409425

Выполнили:
Путинцев Данил
Назин Артем
Группа: P3307
Преподаватель:
Максимов Андрей Николаевич

Санкт-Петербург, 2026 год

Текст задания

Лабораторные работы

Лабораторная работа №1

Введите вариант: 409425

Внимание! У разных вариантов разный текст задания!

Цель работы - на выделенном узле создать и сконфигурировать новый кластер БД Postgres, саму БД, табличные пространства и новую роль, а также произвести наполнение базы в соответствии с заданием. Отчёт по работе должен содержать все команды по настройке, скрипты, а также измененные строки конфигурационных файлов.

Способ подключения к узлу из сети Интернет через helios:
`ssh -J sXXXXXX@helios.cs.ifmo.ru:2222 postgresY@pgZZZ`
Способ подключения к узлу из сети факультета:
`ssh postgresY@pgZZZ`
Номер выделенного узла `pgZZZ`, а также логин и пароль для подключения Вам выдаст преподаватель.

Этап 1. Инициализация кластера БД

- Директория кластера: `$HOME/aum71`
- Кодировка: ANSI1251
- Локаль: русская
- Параметры инициализации задать через аргументы команды

Этап 2. Конфигурация и запуск сервера БД

- Способы подключения: 1) Unix-domain сокет в режиме peer; 2) сокет TCP/IP, принимать подключения к любому IP-адресу узла
- Номер порта: `9425`
- Способ аутентификации TCP/IP клиентов: по паролю MD5
- Остальные способы подключений запретить.
- Настроить следующие параметры сервера БД:
  - max_connections
  - shared_buffers
  - temp_buffers
  - work_mem
  - checkpoint_timeout
  - effective_cache_size
  - fsync
  - commit_delay

  Параметры должны быть подобраны в соответствии с аппаратной конфигурацией:
  оперативная память 4ГБ, хранение на твердотельном накопителе (SSD).
- Директория WAL файлов: `$PGDATA/pg_wal`
- Формат лог-файлов: `.csv`
- Уровень сообщений лога: `WARNING`
- Дополнительно логировать: завершение сессий и продолжительность выполнения команд

Этап 3. Дополнительные табличные пространства и наполнение базы

- Создать новые табличные пространства для партицированной таблицы: `$HOME/nof94`, `$HOME/dyn55`
- На основе template0 создать новую базу: `fakeyellowlaw`
- Создать новую роль, предоставить необходимые права, разрешить подключение к базе.
- От имени новой роли (не администратора) произвести наполнение ВСЕХ созданных баз тестовыми наборами данных. ВСЕ табличные пространства должны использоваться по назначению.
- Вывести список всех табличных пространств кластера и содержащиеся в них объекты.

Ход работы. Этап 1. Инициализация кластера БД

```
[postgres0@pg115 ~]$ locale -a | grep ru
ru_RU.CP1251
ru_RU.CP866
ru_RU.ISO8859-5
ru_RU.KOI8-R
ru_RU.UTF-8
[postgres0@pg115 ~]$ initdb -D $HOME/aum71 --locale=ru_RU.CP1251 --encoding=WIN1251
Файлы, относящиеся к этой СУБД, будут принадлежать пользователю "postgres0".
От его имени также будет запускаться процесс сервера.

Кластер баз данных будет инициализирован с локалью "ru_RU.CP1251".
Выбрана конфигурация текстового поиска по умолчанию "russian".

Контроль целостности страниц данных отключён.

создание каталога /var/db/postgres0/aum71... ок
создание подкаталогов... ок
выбирается реализация динамической разделяемой памяти... posix
выбирается значение max_connections по умолчанию... 100
выбирается значение shared_buffers по умолчанию... 128MB
выбирается часовой пояс по умолчанию... Europe/Moscow
создание конфигурационных файлов... ок
выполняется подготовительный скрипт... ок
выполняется заключительная инициализация... ок
сохранение данных на диске... ок

initdb: предупреждение: включение метода аутентификации "trust" для локальных подключений
initdb: подсказка: Другой метод можно выбрать, отредактировав pg_hba.conf или ещё раз запустив initdb с ключом -A, --auth

Готово. Теперь вы можете запустить сервер баз данных:

    pg_ctl -D /var/db/postgres0/aum71 -l файл_журнала start

[postgres0@pg115 ~]$
```

1. Смотрим список доступных русских локалей. Выбираем ru_RU.CP1251 так как у нас кодировка WIN1251
2. Далее выполняем команду initdb -D $HOME/aum71 --locale=ru_RU.CP1251 --encoding=WIN1251 для инициализации кластера БД. После флага -D указывается директория кластера, --locale указывает какую локаль мы устанавливаем, --encoding устанавливает кодировку
3. Как видим из вывода, кластер баз данных успешно инициализовался.

Ход работы. Этап 2. Конфигурация и запуск сервера БД

Для настройки способов подключения/аутентификации для доступа к БД необходимо настроить конфиг pg_hba.conf.

Полученная конфигурация:

```
local   all             all                                     peer

host    all             all             0.0.0.0/0               md5
host    all             all             ::/0                    md5

local   all             all                                     reject
host    all             all             0.0.0.0/0               reject
host    all             all             ::/0                    reject
```

То есть согласно требованиям:

- Способы подключения: 1) Unix-domain сокет в режиме peer; 2) сокет TCP/IP, принимать подключения к любому IP-адресу узла
- Способ аутентификации TCP/IP клиентов: по паролю MD5

Номер порта и остальные параметры настраиваются в postgresql.conf

Строки полученной конфигурации:

port = 9425

max_connections = 150

shared_buffers = 1GB

temp_buffers = 20MB

work_mem = 10MB

checkpoint_timeout = 15min

effective_cache_size = 2GB

fsync = on

commit_delay = 0

Для настройки логов добавлены настройки: log_destination = 'csvlog' и log_min_messages = warning

Логи начали создаваться в формате csv

```
[postgres0@pg115 ~/aum71]$ cd pg_log
[postgres0@pg115 ~/aum71/pg_log]$ ls
postgresql-2026-02-19_011746.csv        postgresql-2026-03-04_223924.csv        postgresql-2026-03-05_000000.csv
postgresql-2026-02-19_011746.log        postgresql-2026-03-04_223924.log        postgresql-2026-03-05_000000.log
[postgres0@pg115 ~/aum71/pg_log]$
```

Для дополнительного логирования добавим параметры в postgresql.conf:

log_connections = on

log_disconnections = on

log_duration = on

Расширенные логи видны в csv файлах:

```
[postgres0@pg115 ~/aum71/pg_log]$ cat  postgresql-2026-03-06_011728.csv
2026-03-06 01:17:28.687 MSK,,,63581,,69aa00f8.f85d,1,,2026-03-06 01:17:28 MSK,,0,СООБЩЕНИЕ,00000,"завершение вывода в stderr",,"В дальнейшем протокол будет выводиться в ""csvlog"".",,,,,,,,"","postmaster",,0
2026-03-06 01:17:28.687 MSK,,,63581,,69aa00f8.f85d,2,,2026-03-06 01:17:28 MSK,,0,СООБЩЕНИЕ,00000,"запускается PostgreSQL 16.4 on amd64-portbld-freebsd14.1, compiled by FreeBSD clang version 18.1.6 (https://git
hub.com/llvm/llvm-project.git llvmorg-18.1.6-0-g1118c2e05e67), 64-bit",,,,,,,,,,"","postmaster",,0
2026-03-06 01:17:28.688 MSK,,,63581,,69aa00f8.f85d,3,,2026-03-06 01:17:28 MSK,,0,СООБЩЕНИЕ,00000,"для приёма подключений по адресу IPv6 ""::1"" открыт порт 9425",,,,,,,,,,"","postmaster",,0
2026-03-06 01:17:28.688 MSK,,,63581,,69aa00f8.f85d,4,,2026-03-06 01:17:28 MSK,,0,СООБЩЕНИЕ,00000,"для приёма подключений по адресу IPv4 ""127.0.0.1"" открыт порт 9425",,,,,,,,,,"","postmaster",,0
2026-03-06 01:17:28.719 MSK,,,63581,,69aa00f8.f85d,5,,2026-03-06 01:17:28 MSK,,0,СООБЩЕНИЕ,00000,"для приёма подключений открыт Unix-сокет ""/tmp/.s.PGSQL.9425""",,,,,,,,,,"","postmaster",,0
2026-03-06 01:17:28.918 MSK,,,63585,,69aa00f8.f861,1,,2026-03-06 01:17:28 MSK,,0,СООБЩЕНИЕ,00000,"система БД была выключена: 2026-03-05 05:00:00 MSK",,,,,,,,,,"","startup",,0
2026-03-06 01:17:29.142 MSK,,,63581,,69aa00f8.f85d,6,,2026-03-06 01:17:28 MSK,,0,СООБЩЕНИЕ,00000,"система БД готова принимать подключения",,,,,,,,,,"","postmaster",,0
2026-03-06 01:17:36.894 MSK,,,63673,"[local]",69aa0100.f8b9,1,"",2026-03-06 01:17:36 MSK,,0,СООБЩЕНИЕ,00000,"принято подключение: узел=[local]",,,,,,,,,,"","not initialized",,0
2026-03-06 01:17:36.920 MSK,"postgres0","postges",63673,"[local]",69aa0100.f8b9,2,"authentication",2026-03-06 01:17:36 MSK,3/1,0,СООБЩЕНИЕ,00000,"соединение аутентифицировано: идентификатор=""postgres0"" метод
=peer (/var/db/postgres0/aum71/pg_hba.conf:117)",,,,,,,,,,"","client backend",,0
2026-03-06 01:17:36.920 MSK,"postgres0","postges",63673,"[local]",69aa0100.f8b9,3,"authentication",2026-03-06 01:17:36 MSK,3/1,0,СООБЩЕНИЕ,00000,"подключение авторизовано: пользователь=postgres0 база=postges п
риложение=psql",,,,,,,,,,"","client backend",,0
2026-03-06 01:17:36.921 MSK,"postgres0","postges",63673,"[local]",69aa0100.f8b9,4,"startup",2026-03-06 01:17:36 MSK,3/1,0,ВАЖНО,3D000,"база данных ""postges"" не существует",,,,,,,,,,"","client backend",,0
2026-03-06 01:17:41.635 MSK,,,63700,"[local]",69aa0105.f8d4,1,"",2026-03-06 01:17:41 MSK,,0,СООБЩЕНИЕ,00000,"принято подключение: узел=[local]",,,,,,,,,,"","not initialized",,0
2026-03-06 01:17:41.636 MSK,"postgres0","postgres",63700,"[local]",69aa0105.f8d4,2,"authentication",2026-03-06 01:17:41 MSK,3/2,0,СООБЩЕНИЕ,00000,"соединение аутентифицировано: идентификатор=""postgres0"" мето
д=peer (/var/db/postgres0/aum71/pg_hba.conf:117)",,,,,,,,,,"","client backend",,0
2026-03-06 01:17:41.636 MSK,"postgres0","postgres",63700,"[local]",69aa0105.f8d4,3,"authentication",2026-03-06 01:17:41 MSK,3/2,0,СООБЩЕНИЕ,00000,"подключение авторизовано: пользователь=postgres0 база=postgres
 приложение=psql",,,,,,,,,,"","client backend",,0
2026-03-06 01:17:44.332 MSK,"postgres0","postgres",63700,"[local]",69aa0105.f8d4,4,"SELECT",2026-03-06 01:17:41 MSK,3/0,0,СООБЩЕНИЕ,00000,"◆◆◆◆◆◆◆◆◆◆◆◆◆◆◆◆◆: 191.336 ◆◆",,,,,,,,,,"psql","client backend",,0
2026-03-06 01:17:49.952 MSK,"postgres0","postgres",63700,"[local]",69aa0105.f8d4,5,"SELECT",2026-03-06 01:17:41 MSK,3/0,0,СООБЩЕНИЕ,00000,"◆◆◆◆◆◆◆◆◆◆◆◆◆◆◆◆◆: 19.906 ◆◆",,,,,,,,,,"psql","client backend",,0
2026-03-06 01:17:53.381 MSK,"postgres0","postgres",63700,"[local]",69aa0105.f8d4,6,"SELECT",2026-03-06 01:17:41 MSK,3/0,0,СООБЩЕНИЕ,00000,"◆◆◆◆◆◆◆◆◆◆◆◆◆◆◆◆◆: 0.390 ◆◆",,,,,,,,,,"psql","client backend",,0
2026-03-06 01:17:56.336 MSK,"postgres0","postgres",63700,"[local]",69aa0105.f8d4,7,"SELECT",2026-03-06 01:17:41 MSK,3/0,0,СООБЩЕНИЕ,00000,"◆◆◆◆◆◆◆◆◆◆◆◆◆◆◆◆◆: 1.176 ◆◆",,,,,,,,,,"psql","client backend",,0
2026-03-06 01:18:07.870 MSK,"postgres0","postgres",63700,"[local]",69aa0105.f8d4,8,"idle",2026-03-06 01:17:41 MSK,,0,СООБЩЕНИЕ,00000,"◆◆◆◆◆◆◆◆◆◆: ◆◆◆◆◆ ◆◆◆◆◆◆: 0:00:26.235 ◆◆◆◆◆◆◆◆◆◆◆◆◆=postgres0 ◆◆◆◆ ◆◆◆◆◆◆=po
stgres ◆◆◆◆◆◆◆◆◆◆=[local]",,,,,,,,,,"psql","client backend",,0
[postgres0@pg115 ~/aum71/pg_log]$
```

Ход работы. Этап 3. Дополнительные табличные пространства и наполнение базы

Создание новых табличных пространств для партицированной таблицы

$HOME/nof94, $HOME/dyn55

mkdir -p $HOME/nof94

mkdir -p $HOME/dyn55

CREATE TABLESPACE dyn55 LOCATION '/var/db/postgres0/dyn55';

CREATE TABLESPACE nof94 LOCATION '/var/db/postgres0/nof94';

На основе template0 создадим новую базу данных: fakeyellowlaw

CREATE DATABASE fakeyellowlaw TEMPLATE template0;

Создадим новую роль, предоставив необходимые права, разрешим подключение к базе

CREATE ROLE my_user;

ALTER ROLE my_user WITH LOGIN PASSWORD '1111';

GRANT CONNECT ON DATABASE fakeyellowlaw TO my_user;

GRANT ALL ON SCHEMA public TO my_user;

GRANT CREATE ON TABLESPACE nof94 TO my_user;

GRANT CREATE ON TABLESPACE dyn55 TO my_user;

```
postgres=# CREATE ROLE my_user;
CREATE ROLE
postgres=# GRANT CONNECT ON DATABASE fakeyellowlaw TO my_user;
GRANT
postgres=# \c fakeyellowlaw
Вы подключены к базе данных "fakeyellowlaw" как пользователь "postgres0".
fakeyellowlaw=# GRANT ALL ON SCHEMA public TO my_user;
GRANT
fakeyellowlaw=# GRANT CREATE ON TABLESPACE nof94 TO my_user;
GRANT
fakeyellowlaw=# GRANT CREATE ON TABLESPACE dyn55 TO my_user;
GRANT
```

От имени новой роли (не администратора) произвести наполнение ВСЕХ созданных баз тестовыми наборами данных. ВСЕ табличные пространства должны использоваться по назначению.

```
CREATE TYPE group_enum AS ENUM('Positive', 'Negative', 'NULL');

CREATE TYPE issue_enum AS ENUM('Resolved', 'Relevant', 'Not resolved', 'NULL');

CREATE TYPE type_enum AS ENUM('Resolved', 'Relevant', 'Not resolved', 'NULL');


CREATE TABLE TYPE_INFO(

    ID SERIAL PRIMARY KEY,

    NAME varchar(255) NOT NULL,

    STATUS type_enum NOT NULL) TABLESPACE nof94;


CREATE TABLE ISSUE(

    ID SERIAL PRIMARY KEY,

    NAME varchar(16383) UNIQUE,

    STATUS issue_enum) TABLESPACE nof94;


CREATE TABLE SCIENCE(

    ID SERIAL PRIMARY KEY,

    NAME varchar(255) UNIQUE NOT NULL,

    TYPE_ID integer REFERENCES TYPE_INFO NOT NULL) TABLESPACE dyn55;


CREATE TABLE GROUP_INFO(

    ID SERIAL PRIMARY KEY,
```

```
    STATUS group_enum,
    ISSUE_ID integer REFERENCES ISSUE) TABLESPACE dyn55;

CREATE TABLE PERSON(
    ID SERIAL PRIMARY KEY,
    NAME varchar(255) NOT NULL,
    SURNAME varchar(255) NOT NULL) TABLESPACE nof94;

CREATE TABLE DEBATE(
        ID SERIAL PRIMARY KEY,
        SCIENCE_ID integer REFERENCES SCIENCE NOT NULL,
        POSITIVE_GROUP_ID integer REFERENCES GROUP_INFO NOT NULL,
        NEGATIVE_GROUP_ID integer REFERENCES GROUP_INFO NOT NULL,
        ISSUE_ID integer REFERENCES ISSUE NOT NULL,
        CHECK(POSITIVE_GROUP_ID != NEGATIVE_GROUP_ID));

CREATE TABLE GROUP_PERSON(
        ID SERIAL PRIMARY KEY,
        PERSON_ID integer REFERENCES PERSON NOT NULL,
        GROUP_ID integer REFERENCES GROUP_INFO NOT NULL);

INSERT INTO TYPE_INFO VALUES(DEFAULT, 'атомная', 'Resolved');
INSERT INTO TYPE_INFO VALUES(DEFAULT, 'генетика', 'Not resolved');
INSERT INTO TYPE_INFO VALUES(DEFAULT, 'веб', 'Relevant');

INSERT INTO SCIENCE VALUES(DEFAULT, 'физика', 1);
INSERT INTO SCIENCE VALUES(DEFAULT, 'биология', 2);
INSERT INTO SCIENCE VALUES(DEFAULT, 'информатика', 3);
```

```
INSERT INTO ISSUE VALUES(DEFAULT, 'бабах или не бабах?', 'Resolved');
INSERT INTO ISSUE VALUES(DEFAULT, 'может ли родить мужчина?', 'Relevant');
INSERT INTO ISSUE VALUES(DEFAULT, 'все ли мы будем перекладывать jsonы?',
'Relevant');

INSERT INTO GROUP_INFO VALUES(DEFAULT, 'Positive', 1);
INSERT INTO GROUP_INFO VALUES(DEFAULT, 'Negative', 1);
INSERT INTO GROUP_INFO VALUES(DEFAULT, 'Positive', 2);
INSERT INTO GROUP_INFO VALUES(DEFAULT, 'Negative', 2);
INSERT INTO GROUP_INFO VALUES(DEFAULT, 'Positive', 3);
INSERT INTO GROUP_INFO VALUES(DEFAULT, 'Negative', 3);

INSERT INTO PERSON VALUES(DEFAULT, 'Иван', 'Лопатин');
INSERT INTO PERSON VALUES(DEFAULT, 'Данил', 'Путинцев');
INSERT INTO PERSON VALUES(DEFAULT, 'Милана', 'Александрова');
INSERT INTO PERSON VALUES(DEFAULT, 'Дмитрий', 'Карпов');
INSERT INTO PERSON VALUES(DEFAULT, 'Алексей', 'Пастор');
INSERT INTO PERSON VALUES(DEFAULT, 'Екатерина', 'Трифанова');
INSERT INTO PERSON VALUES(DEFAULT, 'Артем', 'Назин');
INSERT INTO PERSON VALUES(DEFAULT, 'Мария', 'Брель');
INSERT INTO PERSON VALUES(DEFAULT, 'Игорь', 'Булинин');
INSERT INTO PERSON VALUES(DEFAULT, 'Пикачу', 'Письмак');
INSERT INTO PERSON VALUES(DEFAULT, 'ПСЖ', 'Балакшин');
INSERT INTO PERSON VALUES(DEFAULT, 'Покемон', 'Клименков');

INSERT INTO GROUP_PERSON VALUES(DEFAULT, 1, 1);
INSERT INTO GROUP_PERSON VALUES(DEFAULT, 1, 3);
INSERT INTO GROUP_PERSON VALUES(DEFAULT, 2, 4);
INSERT INTO GROUP_PERSON VALUES(DEFAULT, 2, 6);
```

```
INSERT INTO GROUP_PERSON VALUES(DEFAULT, 2, 1);
INSERT INTO GROUP_PERSON VALUES(DEFAULT, 3, 2);
INSERT INTO GROUP_PERSON VALUES(DEFAULT, 3, 4);
INSERT INTO GROUP_PERSON VALUES(DEFAULT, 3, 5);
INSERT INTO GROUP_PERSON VALUES(DEFAULT, 4, 2);
INSERT INTO GROUP_PERSON VALUES(DEFAULT, 4, 3);
INSERT INTO GROUP_PERSON VALUES(DEFAULT, 4, 5);
INSERT INTO GROUP_PERSON VALUES(DEFAULT, 5, 2);
INSERT INTO GROUP_PERSON VALUES(DEFAULT, 5, 4);
INSERT INTO GROUP_PERSON VALUES(DEFAULT, 5, 6);
INSERT INTO GROUP_PERSON VALUES(DEFAULT, 6, 1);
INSERT INTO GROUP_PERSON VALUES(DEFAULT, 6, 3);
INSERT INTO GROUP_PERSON VALUES(DEFAULT, 7, 5);
INSERT INTO GROUP_PERSON VALUES(DEFAULT, 8, 3);
INSERT INTO GROUP_PERSON VALUES(DEFAULT, 8, 5);
INSERT INTO GROUP_PERSON VALUES(DEFAULT, 9, 2);
INSERT INTO GROUP_PERSON VALUES(DEFAULT, 10, 4);
INSERT INTO GROUP_PERSON VALUES(DEFAULT, 11, 3);
INSERT INTO GROUP_PERSON VALUES(DEFAULT, 12, 3);


INSERT INTO DEBATE VALUES(DEFAULT, 1, 1, 2, 1);
INSERT INTO DEBATE VALUES(DEFAULT, 2, 3, 4, 2);
INSERT INTO DEBATE VALUES(DEFAULT, 3, 5, 6, 3);
```

Вывести список всех табличных пространств кластера и содержащиеся в них объекты

```
SELECT
  ts.spcname AS tablespace,
  n.nspname AS schema,
```

```
  c.relname AS object_name,
  CASE c.relkind
    WHEN 'r' THEN 'table'
    WHEN 'i' THEN 'index'
    WHEN 'p' THEN 'partitioned table'
    ELSE 'other'
  END AS object_type
FROM pg_class c
JOIN pg_namespace n ON n.oid = c.relnamespace
LEFT JOIN pg_tablespace ts ON ts.oid = c.reltablespace
ORDER BY ts.spcname, n.nspname, c.relname;
```

```
Вы подключены к базе данных "fakeyellowlaw" как пользователь "postgres0".
fakeyellowlaw=# SELECT
    ts.spcname AS tablespace,
    n.nspname AS schema,
    c.relname AS object_name,
    CASE c.relkind
        WHEN 'r' THEN 'table'
        WHEN 'i' THEN 'index'
        WHEN 'p' THEN 'partitioned table'
        ELSE 'other'
    END AS object_type
FROM pg_class c
JOIN pg_namespace n ON n.oid = c.relnamespace
LEFT JOIN pg_tablespace ts ON ts.oid = c.reltablespace
ORDER BY ts.spcname, n.nspname, c.relname;
 tablespace |       schema       |                 object_name                 | object_type
------------+--------------------+---------------------------------------------+-------------
 dyn55      | public             | group_info                                  | table
 dyn55      | public             | science                                     | table
 nof94      | pg_toast           | pg_toast_16551                              | other
 nof94      | pg_toast           | pg_toast_16551_index                        | index
 nof94      | public             | issue                                       | table
 nof94      | public             | person                                      | table
 nof94      | public             | type_info                                   | table
 pg_global  | pg_catalog         | pg_auth_members                             | table
 pg_global  | pg_catalog         | pg_auth_members_grantor_index               | index
 pg_global  | pg_catalog         | pg_auth_members_member_role_index           | index
 pg_global  | pg_catalog         | pg_auth_members_oid_index                   | index
 pg_global  | pg_catalog         | pg_auth_members_role_member_index           | index
 pg_global  | pg_catalog         | pg_authid                                   | table
 pg_global  | pg_catalog         | pg_authid_oid_index                         | index
 pg_global  | pg_catalog         | pg_authid_rolname_index                     | index
 pg_global  | pg_catalog         | pg_database                                 | table
 pg_global  | pg_catalog         | pg_database_datname_index                   | index
 pg_global  | pg_catalog         | pg_database_oid_index                       | index
 pg_global  | pg_catalog         | pg_db_role_setting                          | table
 pg_global  | pg_catalog         | pg_db_role_setting_databaseid_rol_index     | index
 pg_global  | pg_catalog         | pg_parameter_acl                            | table
 pg_global  | pg_catalog         | pg_parameter_acl_oid_index                  | index
 pg_global  | pg_catalog         | pg_parameter_acl_parname_index              | index
 pg_global  | pg_catalog         | pg_replication_origin                       | table
 pg_global  | pg_catalog         | pg_replication_origin_roiident_index        | index
 pg_global  | pg_catalog         | pg_replication_origin_roname_index          | index
 pg_global  | pg_catalog         | pg_shdepend                                 | table
 pg_global  | pg_catalog         | pg_shdepend_depender_index                  | index
 pg_global  | pg_catalog         | pg_shdepend_reference_index                 | index
```

Выводы

В ходе выполнения данной лабораторной работы, мы узнали как инициализировать кластер баз данных PostgreSQL, познакомились с параметрами настройки, с

конфигурационными файлами pg_hba.conf и postgresql.conf, создали табличные пространства и наполнили их данными